CO4 & CO5 – Graphs, Trees and Graph Coloring

Detailed Lecture Notes for UG Students

Contents

1 Introduction to Graphs **3**

1.1 Topics Covered . . . 3
1.2 Definition of a Graph . . . 4
1.3 Basic Terminology . . . 4
1.4 Degree Sequence . . . 5
1.5 Algorithm to Test Whether a Given Vector Is Graphical . . . 6
1.6 Types of Graphs . . . 8
1.7 Subgraphs . . . 9
1.8 Graph Complement . . . 9

2 Graph Isomorphism **9**

2.1 Definition and Properties . . . 9

3 Walks, Paths, Trails, and Circuits **12**

3.1 Basic Definitions . . . 12
3.2 Connected and Disconnected Graphs . . . 12
3.3 Euler Graphs . . . 13
3.4 Hamiltonian Paths and Circuits . . . 13

4 Matrix Representations of Graphs **13**

4.1 Adjacency Matrix . . . 13
4.2 Incidence Matrix . . . 14
4.3 Adjacency Matrix for Directed Graphs . . . 14

5 Shortest Path Algorithms for Directed Graphs **15**

5.1 Dijkstra's Algorithm . . . 15
5.2 Floyd-Warshall Algorithm . . . 16

6 Trees and Their Properties **16**

6.1 Definition and Basic Properties . . . 16
6.2 Leaves and Pendant Vertices . . . 17
6.3 Distance and Center of a Tree . . . 17
6.4 Rooted Trees and Binary Trees . . . 18

7 Spanning Trees **19**
7.1 Definition and Existence 19
7.2 Weighted Graphs and Minimum Spanning Trees 19
7.3 Kruskal's Algorithm 19
7.4 Prim's Algorithm 20

8 Cut Sets and Fundamental Circuits **21**
8.1 Cut Sets 21
8.2 Properties of Cut Sets 21
8.3 Fundamental Circuits 21

9 Graph Coloring **21**
9.1 Proper Coloring 21
9.2 Edge Coloring 22
9.3 Chromatic Polynomial 22
9.4 Applications of Graph Coloring 23

10 Solved Examples **24**

11 Summary **24**

12 Excercise **48**

1 Introduction to Graphs

1.1 Topics Covered

- Fundamentals of graphs and digraphs
- Degree sequences
- Subgraphs
- Special graph types (bipartite, regular, complete)
- Graph isomorphism
- Walks, paths, trails, and circuits
- Connectivity and components
- Graph complements
- Matrix representations of graphs
- Shortest path algorithms

Learning Objectives for CO4

- Understand the fundamental concepts of graphs and digraphs
- Develop skills in analyzing graph properties and structures
- Learn matrix representations and their applications
- Master shortest path algorithms for directed graphs
- Apply graph theory to real-world problem modeling
- Understand connectivity, cycles, and path structures

1.2 Definition of a Graph

Definition 1. *A **graph** $G = (V, E)$ consists of:*

- *A non-empty set V of **vertices** (or nodes)*
- *A set E of **edges**, where each edge connects two distinct vertices*

Example 1. *Let $V = \{1, 2, 3, 4\}$ and $E = \{\{1, 2\}, \{2, 3\}, \{3, 4\}, \{1, 4\}\}$. This represents a graph with 4 vertices and 4 edges forming a cycle.*

Definition 2. *A **directed graph** or **digraph** $G = (V, E)$ is a graph where each edge has a direction associated with it, represented as ordered pairs (u, v).*

1.3 Basic Terminology

Definition 3. *Two vertices u and v are said to be **adjacent** if there exists an edge connecting them.*

Definition 4. *The **degree** of a vertex v, denoted $\deg(v)$, is the number of edges incident to v.*

Definition 5. *A **loop** is an edge that connects a vertex to itself.*

Definition 6. ***Multiple edges** or **parallel edges** are edges that connect the same pair of vertices.*

Definition 7. *A graph with no loops or multiple edges is called a **simple graph**.*

Theorem 1 (Handshaking Lemma)**.** *In any graph $G = (V, E)$, the sum of degrees of all vertices equals twice the number of edges:*

$$\sum_{v \in V} \deg(v) = 2|E|$$

Example 2. *For a graph with vertices $\{1, 2, 3, 4\}$ and edges forming a 4-cycle:*

$$\deg(1) = \deg(2) = \deg(3) = \deg(4) = 2$$

Sum of degrees $= 2 + 2 + 2 + 2 = 8 = 2 \times 4$

Example 3. *Prove that every graph has an even number of odd vertices.*

Proof. Let $G = (V, E)$ be any finite undirected graph.

Suppose the degrees of the vertices of G are

$$d(v_1), d(v_2), d(v_3), \ldots, d(v_n).$$

By the Handshaking Lemma,

$$\sum_{i=1}^{n} d(v_i) = 2|E|.$$

Since $2|E|$ is an even number, the sum of the degrees of all vertices is even.
Now divide the vertices into two groups:

- vertices having even degree,
- vertices having odd degree.

Let the odd degree vertices be

$$u_1, u_2, \ldots, u_k.$$

Then

$$d(u_1) + d(u_2) + \cdots + d(u_k)$$

is the sum of all odd degrees.
The sum of degrees of the even vertices is clearly even.
Therefore,

$$(\text{sum of odd degrees}) + (\text{sum of even degrees})$$

is even.
Since the sum of even degrees is even, the sum of the odd degrees must also be even.
Now observe:

- the sum of an even number of odd integers is even,
- the sum of an odd number of odd integers is odd.

Hence the number of odd degree vertices must be even.
Therefore every graph has an even number of odd vertices.

□

1.4 Degree Sequence

Definition 8. *The **degree sequence** of a graph is the sequence of vertex degrees arranged in non-increasing order.*

Example 4. *For a graph with degrees* $\{2, 3, 1, 4\}$*, the degree sequence is* $(4, 3, 2, 1)$*.*

Theorem 2 (Erdos-Gallai Theorem)**.** *A non-increasing sequence of non-negative integers* $d_1 \geq d_2 \geq \cdots \geq d_n$ *is graphical (i.e., is the degree sequence of some graph) if and only if:*

1. *The sum is even*

2. *For each* $k = 1, 2, \ldots, n$*:*

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$$

Theorem 3 (Hakimi–Havel)**.** *Let*

$$v = [d_1, d_2, d_3, \ldots, d_k]$$

be a nonincreasing vector of k $(k \geq 2)$ *nonnegative integers such that no component*

$$d_i > (k-1).$$

Let v' *be the vector obtained from* v *by deleting* d_1 *and subtracting* 1 *from each of the next* d_1 *components of* v*.*

Let v_1 *be the nonincreasing vector obtained from* v' *by rearranging its components if necessary.*

Then v *is a graphical vector if and only if* v_1 *is a graphical vector.*

1.5 Algorithm to Test Whether a Given Vector Is Graphical

The input is a nonincreasing vector v with integer components.

Step 0. (Initialization) The current vector is v.

Step 1. If the current vector with k components has a component that exceeds $(k-1)$, go to Step 5. Otherwise, go to Step 2.

Step 2. If the current vector has a negative component, go to Step 5. Otherwise, go to Step 3.

Step 3. If the current vector is the zero vector, go to Step 6. Otherwise, go to Step 4.

Step 4. (Iteration) Rearrange the components of the current vector so that it becomes nonincreasing with d_1 as the first component. Delete d_1 from the rearranged vector, and subtract 1 from each of the next d_1 components of the rearranged vector. The vector thus constructed is the updated current vector.
Go to Step 1.

Step 5. The input vector is not graphical. Go to Step 7.

Step 6. The input vector is graphical. Go to Step 7.

Step 7. Stop.

Example 5. *Using the above algorithm it can be verified that*

$$v = [5\ \ 4\ \ 4\ \ 3\ \ 3\ \ 3\ \ 2]$$

is a graphical vector.

Iteration 1:

$$v = [5\ \ 4\ \ 4\ \ 3\ \ 3\ \ 3\ \ 2] \qquad \text{and} \qquad v_1 = [3\ \ 3\ \ 2\ \ 2\ \ 2\ \ 2]$$

Iteration 2:

$$v = [3\ \ 3\ \ 2\ \ 2\ \ 2\ \ 2] \qquad \text{and} \qquad v_1 = [2\ \ 2\ \ 2\ \ 1\ \ 1]$$

Iteration 3:

$$v = [2\ \ 2\ \ 2\ \ 1\ \ 1] \qquad \text{and} \qquad v_1 = [1\ \ 1\ \ 1\ \ 1]$$

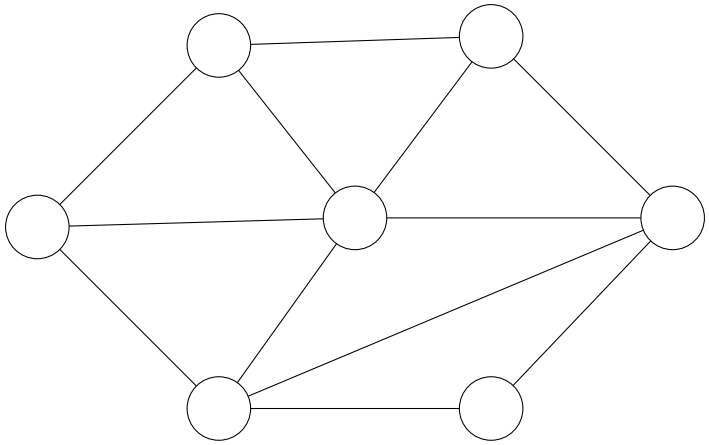
Iteration 4:

$$v = [1\ \ 1\ \ 1\ \ 1] \qquad \text{and} \qquad v_1 = [1\ \ 1\ \ 0]$$

Iteration 5:

$$v = [1\ \ 1\ \ 0] \qquad \text{and} \qquad v_1 = [0\ \ 0]$$

At the end of the fifth iteration we get the zero vector.
Therefore the given vector v is a graphical vector. The graph is given below



1.6 Types of Graphs

Complete Graph

Definition 9. *A **complete graph** K_n is a simple graph with n vertices where every pair of vertices is adjacent.*

Example 6. *In K_4, there are 4 vertices and $\binom{4}{2} = 6$ edges. Each vertex has degree 3.*

Regular Graph

Definition 10. *A graph is k-**regular** if every vertex has degree k.*

Example 7. *The cycle graph C_n is 2-regular for all $n \geq 3$. The complete graph K_n is $(n-1)$-regular.*

Bipartite Graph

Definition 11. *A **bipartite graph** is a graph whose vertices can be partitioned into two disjoint sets X and Y such that every edge connects a vertex in X to a vertex in Y.*

Theorem 4. *A graph is bipartite if and only if it contains no odd cycles.*

Example 8. *The complete bipartite graph $K_{m,n}$ has m vertices in set X and n vertices in set Y, with every vertex in X connected to every vertex in Y. It has mn edges.*

Path and Cycle Graphs

Definition 12. *The **path graph** P_n is a graph with n vertices and $n-1$ edges forming a simple path.*

Definition 13. *The **cycle graph** C_n is a graph with n vertices and n edges forming a simple cycle.*

1.7 Subgraphs

Definition 14. *A **subgraph** $G' = (V', E')$ of a graph $G = (V, E)$ is a graph where $V' \subseteq V$ and $E' \subseteq E$, and each edge in E' connects two vertices in V'.*

Definition 15. *A **spanning subgraph** is a subgraph with the same vertex set as the original graph.*

Definition 16. *An **induced subgraph** on a subset V' of vertices is a subgraph with vertex set V' and edge set containing all edges of G between vertices in V'.*

Example 9. *Let G have vertices $\{1, 2, 3, 4\}$ and edges $\{\{1, 2\}, \{2, 3\}, \{3, 4\}, \{1, 4\}\}$. The induced subgraph on $\{1, 2, 3\}$ has the single edge $\{1, 2\}, \{2, 3\}$.*

1.8 Graph Complement

Definition 17. *The **complement** $\overline{G}$ of a simple graph $G = (V, E)$ is a graph with the same vertex set V, where two vertices are adjacent in $\overline{G}$ if and only if they are not adjacent in G.*

Example 10. *If $G = K_4$ (complete graph on 4 vertices), then $\overline{G}$ has 4 vertices and no edges (empty graph).*

If $G = C_4$ (4-cycle), then $\overline{G}$ is also a 4-cycle (a different one).

2 Graph Isomorphism

2.1 Definition and Properties

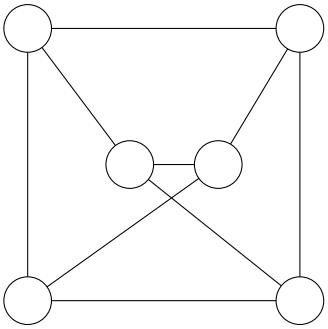
Definition 18. *Two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are **isomorphic**, denoted $G_1 \cong G_2$, if there exists a bijection $\phi : V_1 \to V_2$ such that $(u, v) \in E_1$ if and only if $(\phi(u), \phi(v)) \in E_2$.*

Note 1. *The mapping ϕ is called an **isomorphism**. It preserves adjacency relationships between vertices.*

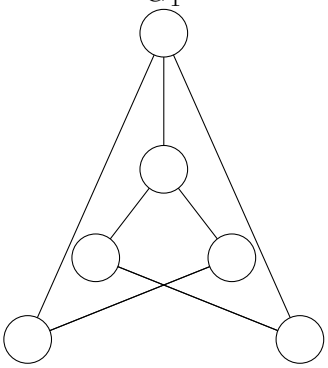
Theorem 5. *If two graphs are isomorphic, then they have:*

- *The same number of vertices*
- *The same number of edges*
- *The same degree sequence*
- *The same number of cycles of each length*

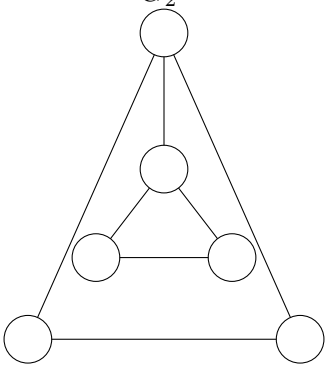
Example 11. *Determine whether the three graphs given in Figure below are isomorphic.*



G_1



G_2



G_3

Let the three graphs be denoted by G_1, G_2 and G_3 respectively.

We compare the graphs using the degrees of their vertices.

Graph G_1:

- The four outer vertices each have degree 3.
- The two inner vertices also have degree 3.

Hence the degree sequence of G_1 is

$$(3, 3, 3, 3, 3, 3).$$

Graph G_2:

- The top vertex has degree 3.
- The two bottom vertices each have degree 3.
- The three inner vertices each have degree 3.

Hence the degree sequence of G_2 is

$$(3, 3, 3, 3, 3, 3).$$

Graph G_3:

- The top vertex has degree 3.
- The two bottom vertices each have degree 3.
- The three inner vertices each have degree 3.

Hence the degree sequence of G_3 is

$$(3, 3, 3, 3, 3, 3).$$

Thus all three graphs have the same degree sequence.

However, equality of degree sequences alone does not guarantee isomorphism.

- In G_1, the graph contains a square as the outer cycle.
- In G_2, the graph is based on a triangular arrangement with crossed internal connections.
- In G_3, the three inner vertices themselves form a triangle.

Observe that G_3 contains an additional triangular subgraph not present in the same way in G_1 and G_2.

Therefore G_3 cannot be isomorphic to either G_1 or G_2.

Also, the cycle structures of G_1 and G_2 are different, so they are not isomorphic.

$$\therefore \qquad G_1,\ G_2,\ \text{and } G_3 \text{ are not isomorphic.}$$

3 Walks, Paths, Trails, and Circuits

3.1 Basic Definitions

Definition 19. *A **walk** in a graph is a sequence of vertices $v_0, v_1, \ldots, v_k$ where each consecutive pair is connected by an edge. The **length** of the walk is k (the number of edges).*

Definition 20. *A **trail** is a walk where all edges are distinct (no repeated edges).*

Definition 21. *A **path** is a walk where all vertices are distinct (which implies all edges are distinct).*

Definition 22. *A **circuit** (or **closed walk**) is a walk that starts and ends at the same vertex.*

Definition 23. *A **cycle** is a circuit where all vertices except the first and last are distinct. A cycle of length n is denoted C_n.*

Note 2. *Every path of length ≥ 1 is a trail, but not every trail is a path (trails can repeat vertices but not edges).*

Example 12. *In a graph with vertices $\{1, 2, 3, 4\}$ and edges forming a 4-cycle:*

- *Walk:* $1 \to 2 \to 3 \to 4 \to 1 \to 2$ *(length 5)*
- *Path:* $1 \to 2 \to 3 \to 4$ *(length 3, all vertices distinct)*
- *Cycle:* $1 \to 2 \to 3 \to 4 \to 1$ *(length 4)*
- *Trail:* $1 \to 2 \to 3 \to 4 \to 1 \to 2$ *(all edges distinct but vertex 1, 2 repeat)*

3.2 Connected and Disconnected Graphs

Definition 24. *Two vertices u and v in a graph are **connected** if there exists a path from u to v.*

Definition 25. *A graph is **connected** if every pair of vertices is connected.*

Definition 26. *A **connected component** of a graph is a maximal connected subgraph.*

Example 13. *A graph with vertices $\{1, 2, 3, 4, 5\}$ and edges $\{\{1, 2\}, \{2, 3\}, \{4, 5\}\}$ has two connected components:*

- *Component 1: vertices $\{1, 2, 3\}$*
- *Component 2: vertices $\{4, 5\}$*

Definition 27. *A **disconnected graph** has more than one connected component.*

3.3 Euler Graphs

Definition 28. *An **Euler path** is a trail that uses every edge of the graph exactly once.*

Definition 29. *An **Euler circuit** is a circuit that uses every edge of the graph exactly once.*

Definition 30. *A **Eulerian graph** is a connected graph that contains an Euler circuit.*

Theorem 6 (Euler's Theorem)**.** *A connected graph G has an Euler circuit if and only if every vertex has even degree.*

Theorem 7. *A connected graph G has an Euler path (but not a circuit) if and only if it has exactly two vertices of odd degree.*

Example 14. *Consider a graph with vertices $\{1, 2, 3, 4\}$ and edges forming a 4-cycle. Each vertex has degree 2 (even), so the graph has an Euler circuit: $1 \to 2 \to 3 \to 4 \to 1$.*

Example 15. *The complete graph K_5 has all vertices with degree 4 (even), so it is Eulerian and has Euler circuits.*

3.4 Hamiltonian Paths and Circuits

Definition 31. *A **Hamiltonian path** is a path that visits every vertex of the graph exactly once.*

Definition 32. *A **Hamiltonian circuit** is a cycle that visits every vertex of the graph exactly once.*

Definition 33. *A **Hamiltonian graph** is a graph that contains a Hamiltonian circuit.*

Note 3. *Unlike Euler paths/circuits, there is no simple criterion for determining whether a graph has a Hamiltonian path or circuit. This is an NP-complete problem in general.*

Theorem 8 (Dirac's Theorem)**.** *If G is a simple graph with $n \geq 3$ vertices where every vertex has degree at least $\frac{n}{2}$, then G has a Hamiltonian circuit.*

Example 16. *In a 4-cycle (vertices 1, 2, 3, 4), the Hamiltonian circuit is $1 \to 2 \to 3 \to 4 \to 1$.*

4 Matrix Representations of Graphs

4.1 Adjacency Matrix

Definition 34. *The **adjacency matrix** A of a graph $G = (V, E)$ with vertices $v_1, v_2, \ldots, v_n$ is an $n \times n$ matrix where:*

$$A_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \in E \\ 0 & \text{otherwise} \end{cases}$$

Note 4. *For an undirected graph, the adjacency matrix is symmetric: $A_{ij} = A_{ji}$. For a directed graph, this symmetry need not hold.*

Example 17. *For a 3-cycle with vertices $\{1, 2, 3\}$ and edges $\{\{1, 2\}, \{2, 3\}, \{1, 3\}\}$:*

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

Theorem 9. *The (i, j)-entry of A^k (where A is the adjacency matrix) gives the number of walks of length k from vertex i to vertex j.*

Example 18. *For the adjacency matrix above:*

$$A^2 = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}$$

The entry $A^2_{11} = 2$ means there are 2 walks of length 2 from vertex 1 to itself: $1 \to 2 \to 1$ and $1 \to 3 \to 1$.

4.2 Incidence Matrix

Definition 35. *The **incidence matrix** B of a graph $G = (V, E)$ with vertices $v_1, \ldots, v_n$ and edges $e_1, \ldots, e_m$ is an $n \times m$ matrix where:*

$$B_{ij} = \begin{cases} 1 & \text{if vertex } v_i \text{ is incident to edge } e_j \\ 0 & \text{otherwise} \end{cases}$$

Example 19. *For a triangle with vertices $\{1, 2, 3\}$ and edges $\{e_1 = \{1, 2\}, e_2 = \{2, 3\}, e_3 = \{1, 3\}\}$:*

$$B = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}$$

Note 5. *Each column of the incidence matrix has exactly 2 ones (for a simple graph without loops). The sum of each row equals the degree of the corresponding vertex.*

4.3 Adjacency Matrix for Directed Graphs

Definition 36. *For a directed graph, the adjacency matrix A is defined as:*

$$A_{ij} = \begin{cases} 1 & \text{if there is a directed edge from } v_i \text{ to } v_j \\ 0 & \text{otherwise} \end{cases}$$

Note 6. *For a directed graph, the adjacency matrix is generally not symmetric.*

Example 20. *For a directed graph with vertices* $\{1, 2, 3\}$ *and edges* $\{1 \to 2, 2 \to 3, 3 \to 1\}$*:*

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

5 Shortest Path Algorithms for Directed Graphs

5.1 Dijkstra's Algorithm

Definition 37. ***Dijkstra's Algorithm*** *finds the shortest path from a source vertex to all other vertices in a weighted directed graph with non-negative edge weights.*

Algorithm Steps

1. Initialize: Set distance to source as 0, all other distances as ∞. Mark all vertices as unvisited.
2. While there are unvisited vertices:
   (a) Select the unvisited vertex with minimum distance
   (b) For each unvisited neighbor:
      i. Calculate the distance through the current vertex
      ii. If this distance is less than the recorded distance, update it
   (c) Mark the current vertex as visited

Example 21. *Consider a directed graph with vertices* $\{A, B, C, D\}$ *and weighted edges:*

- $A \to B : 4$
- $A \to C : 2$
- $B \to C : 1$
- $B \to D : 5$
- $C \to D : 8$
- $C \to B : 3$

Running Dijkstra's algorithm from A:

Iteration	*Current*	*A*	*B*	*C*	*D*
Start	–	*0*	∞	∞	∞
1	*A*	*0*	*4*	*2*	∞
2	*C*	*0*	*2*	*2*	*10*
3	*B*	*0*	*2*	*2*	*7*
4	*D*	*0*	*2*	*2*	*7*

Shortest paths from A:

- *A to A: 0*
- *A to B: 2 (via $A \to C \to B$)*
- *A to C: 2 (direct)*
- *A to D: 7 (via $A \to C \to B \to D$)*

5.2 Floyd-Warshall Algorithm

Definition 38. *The **Floyd-Warshall Algorithm** computes shortest paths between all pairs of vertices in a directed graph.*

Algorithm Steps

1. Initialize: Set $dist[i][j]$ to the weight of edge from i to j, or ∞ if no edge exists.
2. For each intermediate vertex k:
   (a) For each pair of vertices (i, j):

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$$

Example 22. *For the same directed graph, the all-pairs shortest path matrix is:*

$$\begin{pmatrix} 0 & 2 & 2 & 7 \\ \infty & 0 & 1 & 5 \\ \infty & 3 & 0 & 8 \\ \infty & \infty & \infty & 0 \end{pmatrix}$$

6 Trees and Their Properties

6.1 Definition and Basic Properties

Definition 39. *A **tree** is a connected acyclic graph.*

Definition 40. *A **forest** is a graph where each connected component is a tree.*

Theorem 10. *The following statements are equivalent for a graph $G = (V, E)$ with n vertices:*

1. *G is a tree*
2. *G is connected and has $n - 1$ edges*
3. *G is acyclic and has $n - 1$ edges*
4. *G is connected and acyclic*
5. *Any two vertices are connected by exactly one path*

Example 23. *A tree with 5 vertices must have exactly 4 edges. A disconnected graph with 5 vertices and 4 edges is a forest (not a tree).*

6.2 Leaves and Pendant Vertices

Definition 41. *A **leaf** (or **pendant vertex**) is a vertex of degree 1.*

Theorem 11. *Every tree with $n \geq 2$ vertices has at least two leaves.*

Example 24. *In a path graph P_5 (vertices 1-2-3-4-5), vertices 1 and 5 are leaves.*

6.3 Distance and Center of a Tree

Definition 42. *The **distance** $d(u, v)$ between two vertices is the length of the unique path connecting them.*

Definition 43. *The **eccentricity** $e(v)$ of a vertex v is the maximum distance from v to any other vertex:*

$$e(v) = \max_{u \in V} d(u, v)$$

Definition 44. *The **radius** $r(T)$ of a tree is the minimum eccentricity:*

$$r(T) = \min_{v \in V} e(v)$$

Definition 45. *The **diameter** $d(T)$ of a tree is the maximum eccentricity:*

$$d(T) = \max_{v \in V} e(v)$$

Definition 46. *A **center** of a tree is a vertex with eccentricity equal to the radius.*

Theorem 12. *Every tree has either one or two centers. A tree has one center if and only if the diameter is even.*

Example 25. *For the path P_5 (vertices 1-2-3-4-5):*

- $e(1) = 4, e(2) = 3, e(3) = 2, e(4) = 3, e(5) = 4$
- *Radius = 2, Diameter = 4*
- *Center = vertex 3 (the unique vertex with eccentricity 2)*

6.4 Rooted Trees and Binary Trees

Definition 47. *A **rooted tree** is a tree in which one vertex (the **root**) is designated as special. All edges are directed away from the root.*

Definition 48. *For a vertex v in a rooted tree:*

- *The **parent** of v is the vertex adjacent to v on the path to the root*
- *The **children** of v are its adjacent vertices that are not its parent*
- *The **depth** of v is the distance from the root*
- *The **height** of a tree is the maximum depth of any vertex*

Definition 49. *A **binary tree** is a rooted tree where each vertex has at most two children.*

Definition 50. *A **complete binary tree** is a binary tree where every internal vertex has exactly two children.*

Definition 51. *A **full binary tree** is a binary tree where every vertex has either 0 or 2 children.*

Note 7. *In a complete binary tree with n internal vertices, there are $n + 1$ leaves.*

Example 26. *A complete binary tree of height 3 has:*

- *1 vertex at depth 0 (root)*
- *2 vertices at depth 1*
- *4 vertices at depth 2*
- *8 vertices at depth 3 (all leaves)*
- *Total: 15 vertices*

7 Spanning Trees

7.1 Definition and Existence

Definition 52. *A **spanning tree** of a connected graph G is a spanning subgraph that is a tree.*

Theorem 13. *Every connected graph has at least one spanning tree.*

Example 27. *For a 4-cycle graph C_4 (vertices 1-2-3-4-1), one spanning tree is the path 1-2-3-4. Another is 1-2-3 with edge from 1 to 4.*

7.2 Weighted Graphs and Minimum Spanning Trees

Definition 53. *A **weighted graph** is a graph where each edge has an associated weight (or cost).*

Definition 54. *The **weight** of a spanning tree is the sum of weights of all edges in the tree.*

Definition 55. *A **Minimum Spanning Tree** (MST) is a spanning tree with minimum weight.*

Note 8. *An MST is unique when all edge weights are distinct.*

7.3 Kruskal's Algorithm

Definition 56. ***Kruskal's Algorithm** constructs an MST by greedily selecting the smallest weight edges that don't form a cycle.*

Algorithm Steps

1. Sort all edges in non-decreasing order of weight
2. Initialize an empty set T of edges
3. For each edge (u, v) in sorted order:
   (a) If adding (u, v) to T does not create a cycle, add it
   (b) Otherwise, skip it
4. Stop when T has $n - 1$ edges

Example 28. *Consider a weighted graph with 4 vertices and edges:*

- $(1, 2) : 1$

- $(2, 3) : 3$
- $(3, 4) : 2$
- $(1, 4) : 4$
- $(1, 3) : 5$

Kruskal's algorithm:

1. *Select edge* $(1, 2) : 1$
2. *Select edge* $(3, 4) : 2$
3. *Select edge* $(2, 3) : 3$ *(adds edge between components)*
4. *Skip edge* $(1, 4) : 4$ *(would create cycle)*
5. *Skip edge* $(1, 3) : 5$ *(would create cycle)*

Resulting MST has edges $\{(1, 2), (3, 4), (2, 3)\}$ *with total weight 6.*

7.4 Prim's Algorithm

Definition 57. ***Prim's Algorithm*** *constructs an MST by starting from an arbitrary vertex and greedily adding the minimum weight edge that connects the tree to a new vertex.*

Algorithm Steps

1. Start with an arbitrary vertex; add it to the tree T
2. While T does not contain all vertices:
   (a) Find the minimum weight edge connecting a vertex in T to a vertex not in T
   (b) Add this edge and the new vertex to T

Example 29. *Using the same weighted graph, starting from vertex 1:*

1. *Start with vertex 1*
2. *Minimum edge from 1 is* $(1, 2) : 1$ *? add vertex 2*
3. *Minimum edge from* $\{1, 2\}$ *is* $(2, 3) : 3$ *? add vertex 3*
4. *Minimum edge from* $\{1, 2, 3\}$ *is* $(3, 4) : 2$ *? add vertex 4*

Resulting MST: edges $\{(1, 2), (2, 3), (3, 4)\}$ *with total weight 6.*

8 Cut Sets and Fundamental Circuits

8.1 Cut Sets

Definition 58. *A **cut set** (or **edge cut**) is a minimal set of edges whose removal disconnects the graph (or increases the number of connected components).*

Definition 59. *An edge is called a **bridge** if its removal increases the number of connected components.*

Note 9. *A bridge is a cut set consisting of a single edge.*

Example 30. *In a 4-cycle (1-2-3-4-1), any single edge is a cut set of the cycle. The set* $\{\{1,2\},\{3,4\}\}$ *is also a cut set.*

8.2 Properties of Cut Sets

Theorem 14. *In a connected graph with* n *vertices and* m *edges, if there are* k *minimal cut sets, then* $n-1 \leq m \leq n(n-1)/2$.

8.3 Fundamental Circuits

Definition 60. *Let* T *be a spanning tree of a connected graph* G*. For each edge* e *not in* T*, the **fundamental circuit** relative to* e *is the unique cycle formed by adding* e *to* T *and removing no edge.*

Note 10. *The set of fundamental circuits forms a basis for the cycle space of the graph.*

Example 31. *For a 4-cycle graph, consider the spanning tree consisting of edges* $\{(1,2),(2,3),(3,4)\}$*. The fundamental circuit relative to the non-tree edge* $(4,1)$ *is* $1-2-3-4-1$.

9 Graph Coloring

9.1 Proper Coloring

Definition 61. *A **proper coloring** of a graph is an assignment of colors to vertices such that no two adjacent vertices have the same color.*

Definition 62. *The **chromatic number** $\chi(G)$ is the minimum number of colors needed for a proper coloring.*

Theorem 15. *For any graph* G*:*

1. $\chi(K_n) = n$ *(complete graph)*

2. $\chi(C_n) = 2$ *if* n *is even;* $\chi(C_n) = 3$ *if* n *is odd*

3. $\chi(G) \leq \Delta(G) + 1$, *where* $\Delta(G)$ *is the maximum degree*

Theorem 16 (Brooks' Theorem)**.** *If* G *is a connected graph that is neither a complete graph nor an odd cycle, then* $\chi(G) \leq \Delta(G)$.

Example 32. *For a 4-cycle* C_4*:*

- *Can be colored with 2 colors (it's bipartite)*
- *So* $\chi(C_4) = 2$

For a 5-cycle C_5*:*

- *Requires 3 colors (it's an odd cycle)*
- *So* $\chi(C_5) = 3$

9.2 Edge Coloring

Definition 63. *An* ***edge coloring*** *of a graph is an assignment of colors to edges such that no two adjacent edges have the same color.*

Definition 64. *The* ***chromatic index*** $\chi'(G)$ *is the minimum number of colors needed for a proper edge coloring.*

Theorem 17 (Vizing's Theorem)**.** *For any simple graph* G*:*

$$\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1$$

where $\Delta(G)$ *is the maximum degree.*

Example 33. *For a 4-cycle:*

- *Maximum degree* $\Delta = 2$
- *Chromatic index* $\chi' = 2$
- *Can be edge-colored with 2 colors (opposite edges get the same color)*

For the complete graph K_4*:*

- *Maximum degree* $\Delta = 3$
- *Chromatic index* $\chi' = 3$

9.3 Chromatic Polynomial

Definition 65. *The* ***chromatic polynomial*** $P(G, k)$ *of a graph* G *is the number of proper* k*-colorings of* G.

Computing Chromatic Polynomials

Theorem 18 (Deletion-Contraction)**.** *For any edge $e = (u, v)$ in graph G:*

$$P(G, k) = P(G - e, k) - P(G/e, k)$$

where:

- *$G - e$ is the graph with edge e removed*
- *G/e is the graph with edge e contracted (vertices u and v merged)*

Example 34. *For the path P_2 (two vertices connected by an edge):*

$$P(P_2, k) = k(k - 1)$$

because the first vertex can be any of k colors, and the second can be any of $k - 1$ colors.

For the path P_3 (three vertices in a line):

$$P(P_3, k) = k(k - 1)^2$$

Example 35. *For a triangle (3-cycle) C_3:*

$$P(C_3, k) = k(k - 1)(k - 2)$$

using deletion-contraction on an edge.

Example 36. *For a 4-cycle C_4:*

$$P(C_4, k) = (k - 1)^4 + (k - 1)$$

which simplifies using deletion-contraction on an edge.

9.4 Applications of Graph Coloring

Graph coloring has numerous real-world applications:

1. **Scheduling Problems**: Assigning time slots to courses such that no two courses have conflicting instructors or rooms
2. **Map Coloring**: Coloring regions of a map so adjacent regions have different colors
3. **Register Allocation**: In compilers, assigning processor registers to variables
4. **Frequency Assignment**: Assigning frequencies to radio transmitters so no interference occurs

5. **Timetabling**: Creating class timetables with no conflicts

6. **Sudoku Puzzles**: Can be modeled as graph coloring problems

10 Solved Examples

Example 37. ***Question:*** *A graph has vertices with degrees 2, 2, 3, 4, 5. Is this degree sequence graphical?*

Solution: *Using the handshaking lemma:*

$$\sum \deg(v) = 2 + 2 + 3 + 4 + 5 = 16 = 2 \times 8$$

The sum is even, which is necessary. For sufficiency, we can construct such a graph:

- *Arrange vertices as:* $v_1(2), v_2(2), v_3(3), v_4(4), v_5(5)$
- *Connect them appropriately to achieve the required degrees*

Yes, this sequence is graphical.

Example 38. ***Question:*** *Does the complete graph* K_5 *have an Euler circuit?*

Solution: *In* K_5*, each vertex has degree* $5 - 1 = 4$ *(even). Since all vertices have even degree and the graph is connected, by Euler's theorem,* K_5 *has an Euler circuit.*

Example 39. ***Question:*** *Find a spanning tree for a square graph (4-cycle) with vertices 1, 2, 3, 4.*

Solution: *One spanning tree is the path 1-2-3-4 using edges* $\{(1,2),(2,3),(3,4)\}$*. Another is the star centered at vertex 1: edges* $\{(1,2),(1,3),(1,4)\}$*. Both have 3 edges and connect all 4 vertices without forming cycles.*

Example 40. ***Question:*** *What is the chromatic number of a bipartite graph?*

Solution: *The chromatic number of any bipartite graph is at most 2, because we can color one partition with color 1 and the other with color 2. Since bipartite graphs have no odd cycles, no vertex within the same partition is adjacent to another in that partition.*

11 Summary

In this module (CO4 and CO5) we have studied:

CO4: Graphs and Matrix Representation

- Fundamental definitions and terminology of graphs and digraphs
- Degree sequence and degree-related properties

- Subgraphs and graph complements
- Graph isomorphism and characterization
- Walks, paths, trails, circuits, and cycles
- Connectivity and connected components
- Euler graphs and Eulerian paths/circuits
- Hamiltonian paths and circuits
- Adjacency and incidence matrices
- Shortest path algorithms (Dijkstra, Floyd-Warshall)

CO5: Trees and Graph Coloring

- Properties and characterizations of trees
- Distance, eccentricity, and centers in trees
- Rooted and binary trees
- Spanning trees and minimum spanning trees
- Kruskal's and Prim's algorithms for MSTs
- Cut sets and fundamental circuits
- Graph coloring and chromatic number
- Edge coloring and chromatic index
- Chromatic polynomials and their computation
- Applications to scheduling and resource allocation

Real-World Applications

These concepts form the mathematical foundation for:

- Computer networking (routing algorithms, network design)
- Operations research (optimization, scheduling)
- Compiler design (register allocation)
- Circuit design and VLSI

- Transportation networks
- Social network analysis
- Bioinformatics (protein networks)
- Cryptography and coding theory

Question-1

How many edges are there in an undirected graph with 10 vertices each of degree 6?

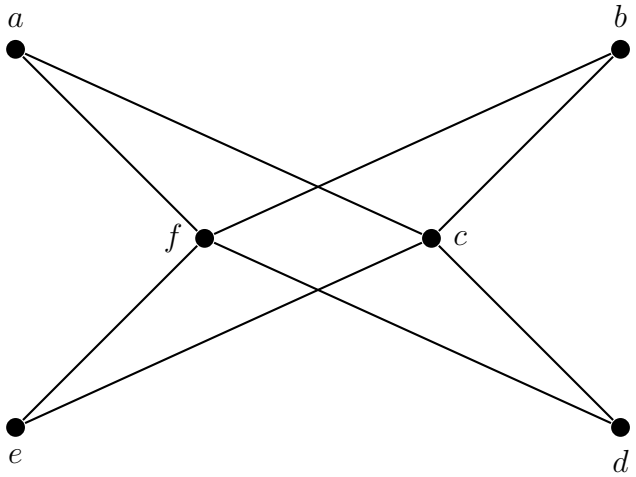
Solution: Let us consider that e be the number edges in the graph $G = (V, E)$, where V is the set of all vertices of the graph G and E be the set of all edges of the same graph G.
Then, by using the **Handshaking Theorem**, we can write

$$\begin{aligned} 2e &= \sum_{v \in V} deg(v) \\ &\Rightarrow 2e = \text{sum of the degrees of all the vertices} \\ &\Rightarrow 2e = 10 \times 6 \\ &\Rightarrow 2e = 60 \\ &\Rightarrow e = 30 \end{aligned}$$

Hence, there are 30 edges in the graph G. □

Question-2

Determine whether the following graph is a bipartite graph.



Solution: **Yes**, the graph is bipartite.

A graph is bipartite if its vertices can be divided into two separate sets (let's call them **Set 1** and **Set 2**) such that every edge in the graph connects a vertex in Set 1 to a vertex in Set 2. Importantly, there can be **no edges** connecting vertices within the same set.

We can test if a graph is bipartite by trying to assign each vertex to one of the two sets:

1. **Start with vertex f:** Place f into **Set 1**.
2. **Assign f's neighbors:** Look at all the vertices connected to f. The graph shows edges connecting f to a, b, e, and d. Because they share an edge with f, they cannot be in the same set as f. Therefore, we place a, b, e, and d into **Set 2**.
3. **Assign the remaining vertex:** The only vertex left to assign is c.
4. **Check c's neighbors:** Look at the edges connected to c. It is connected to a, b, e, and d. Because all of its neighbors are currently in **Set 2**, vertex c must be placed in **Set 1**.

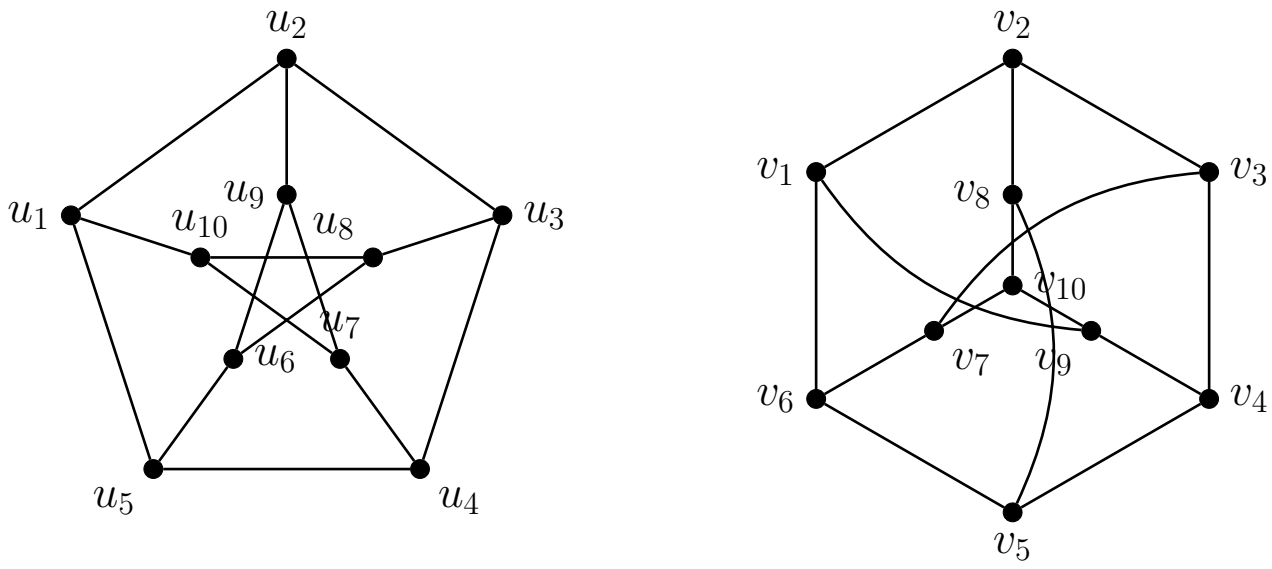
- **Set 1:** $\{f, c\}$
- **Set 2:** $\{a, b, e, d\}$

Because every single edge in the graph runs only between a vertex in Set 1 and a vertex in Set 2, we have successfully partitioned the graph. Therefore, **the graph is bipartite.** □

(Note: Because every vertex in Set 1 is connected to every single vertex in Set 2, this is specifically known as a complete bipartite graph, denoted as $K_{2,4}$.)

Question-3

Determine whether the given pair of graphs is isomorphic. Exhibit an isomorphism or provide a rigorous argument that none exists.



Solution: Based on a detailed structural analysis, **the two graphs are isomorphic.**

Before constructing a mapping, we first confirm that both graphs share identical fundamental properties. Let the left graph be G_1 and the right graph be G_2.

- **Vertices:** Both G_1 and G_2 have exactly 10 vertices ($|V| = 10$).
- **Edges:** Both graphs have exactly 15 edges ($|E| = 15$).
- **Degrees:** Every vertex in G_1 has a degree of 3. By counting the connections in G_2 (including outer edges, straight spokes, curved edges, and inner central connections), every vertex in G_2 also has a degree of 3. Both are therefore 3-regular (cubic) graphs.

To prove isomorphism, we define a bijective function $f : V(G_1) \to V(G_2)$ that perfectly preserves adjacency.

Let the mapping f be defined as follows:

$$\begin{aligned} f(u_1) &= v_9 & f(u_6) &= v_5 \\ f(u_2) &= v_1 & f(u_7) &= v_7 \\ f(u_3) &= v_2 & f(u_8) &= v_8 \\ f(u_4) &= v_3 & f(u_9) &= v_6 \\ f(u_5) &= v_4 & f(u_{10}) &= v_{10} \end{aligned}$$

To rigorously prove this mapping is an isomorphism, we apply the function f to all 15 edges of G_1 and verify that the corresponding mapped edge exists in G_2.

Outer Pentagon Edges of G_1:

- $(u_1, u_2) \to (v_9, v_1)$: Exists in G_2 (Curved edge)

- $(u_2, u_3) \to (v_1, v_2)$: Exists in G_2 (Outer hexagon)
- $(u_3, u_4) \to (v_2, v_3)$: Exists in G_2 (Outer hexagon)
- $(u_4, u_5) \to (v_3, v_4)$: Exists in G_2 (Outer hexagon)
- $(u_5, u_1) \to (v_4, v_9)$: Exists in G_2 (Straight spoke)

Inner Star Edges of G_1:

- $(u_6, u_8) \to (v_5, v_8)$: Exists in G_2 (Curved edge)
- $(u_8, u_{10}) \to (v_8, v_{10})$: Exists in G_2 (Inner central edge)
- $(u_{10}, u_7) \to (v_{10}, v_7)$: Exists in G_2 (Inner central edge)
- $(u_7, u_9) \to (v_7, v_6)$: Exists in G_2 (Straight spoke)
- $(u_9, u_6) \to (v_6, v_5)$: Exists in G_2 (Outer hexagon)

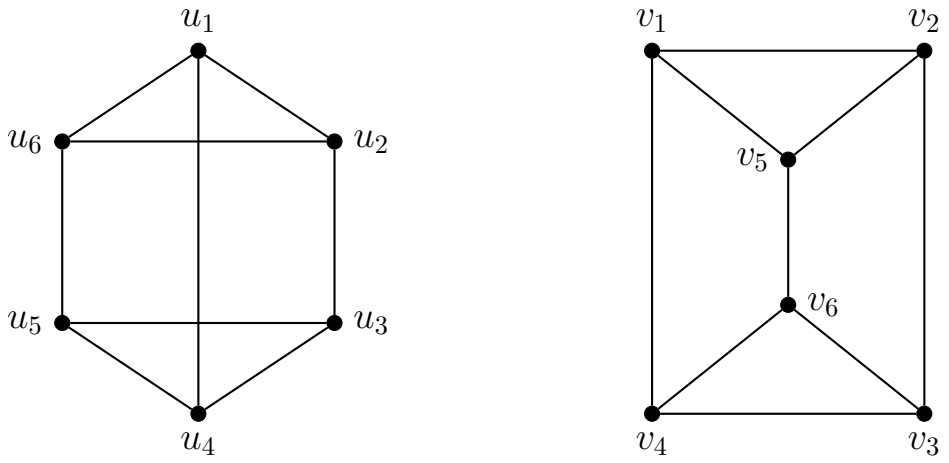
Connecting Spoke Edges of G_1:

- $(u_1, u_{10}) \to (v_9, v_{10})$: Exists in G_2 (Inner central edge)
- $(u_2, u_9) \to (v_1, v_6)$: Exists in G_2 (Outer hexagon)
- $(u_3, u_8) \to (v_2, v_8)$: Exists in G_2 (Straight spoke)
- $(u_4, u_7) \to (v_3, v_7)$: Exists in G_2 (Curved edge)
- $(u_5, u_6) \to (v_4, v_5)$: Exists in G_2 (Outer hexagon)

Since every edge in G_1 corresponds directly to a valid edge in G_2 under the bijective mapping f, the function is a valid isomorphism. Therefore, $G_1 \cong G_2$. $\square$

Question-4

Determine whether the given pair of graphs is isomorphic. Exhibit an isomorphism or provide a rigorous argument that none exists.



Solution: Based on a detailed analysis of the two graphs, we can determine that **the given pair of graphs is indeed isomorphic.**

Here is the rigorous step-by-step mathematical solution and the explicit isomorphism.

Let the left graph be G_1 and the right graph be G_2.

- **Number of Vertices:** $|V(G_1)| = 6$ and $|V(G_2)| = 6$.
- **Vertex Degrees:** By counting the connections, every vertex in G_1 has a degree of 3 (it is a 3-regular graph). Similarly, every vertex in G_2 also has a degree of 3.
- **Number of Edges:** Using the handshaking lemma ($\frac{1}{2}\sum$ degrees), both graphs have $\frac{6\times3}{2} = 9$ edges.

Since the basic invariants match, we can proceed to look at the internal structure.

To find an isomorphism mapping, we need to identify unique structural features, such as specific cycles (loops). Let's look for triangles (3-cycles) in both graphs.

- **Cycles in G_1:** There are exactly two separate 3-cycles: the top triangle formed by $\{u_1, u_2, u_6\}$ and the bottom triangle formed by $\{u_4, u_3, u_5\}$.
- **Cycles in G_2:** Looking closely at G_2, there are also exactly two 3-cycles: the top inner triangle formed by $\{v_1, v_2, v_5\}$ and the bottom inner triangle formed by $\{v_4, v_3, v_6\}$.

Furthermore, in both graphs, the two triangles are connected to each other by exactly three independent edges (a perfect matching).

- In G_1, the connecting edges are (u_1, u_4), (u_2, u_3), and (u_6, u_5).
- In G_2, the connecting edges are (v_1, v_4), (v_2, v_3), and (v_5, v_6).

Because the structural anatomy of both graphs is identical (they are both representations of the triangular prism graph), we can construct a one-to-one mapping between them.

We will define a bijective function $f : V(G_1) \to V(G_2)$ by mapping the vertices of the first triangle in G_1 to the first triangle in G_2, and then following the connecting edges to map the second triangle.

Let's establish the following mapping:

- $f(u_1) = v_1$
- $f(u_2) = v_2$
- $f(u_3) = v_3$
- $f(u_4) = v_4$
- $f(u_5) = v_6$
- $f(u_6) = v_5$

To prove that f is a valid isomorphism, we must verify that for every edge (x, y) in G_1, there is a corresponding edge $(f(x), f(y))$ in G_2.

Let's test all 9 edges of G_1 under our mapping function f:

Edges of the first triangle:

- $(u_1, u_2) \to (f(u_1), f(u_2)) = (v_1, v_2) :$ *Valid edge in* G_2
- $(u_2, u_6) \to (f(u_2), f(u_6)) = (v_2, v_5) :$ *Valid edge in* G_2
- $(u_6, u_1) \to (f(u_6), f(u_1)) = (v_5, v_1) :$ *Valid edge in* G_2

Edges of the second triangle:

- $(u_4, u_3) \to (f(u_4), f(u_3)) = (v_4, v_3) :$ *Valid edge in* G_2
- $(u_3, u_5) \to (f(u_3), f(u_5)) = (v_3, v_6) :$ *Valid edge in* G_2
- $(u_5, u_4) \to (f(u_5), f(u_4)) = (v_6, v_4) :$ *Valid edge in* G_2

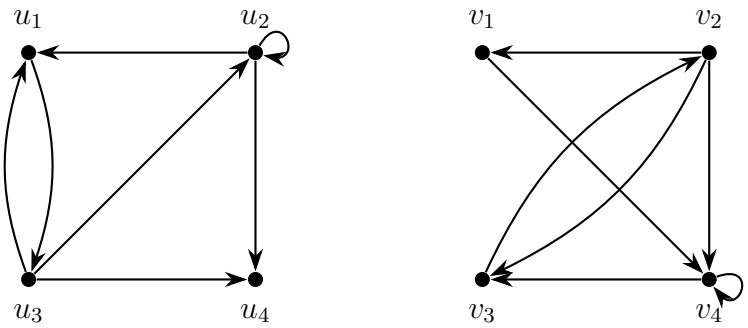
Edges connecting the triangles:

- $(u_1, u_4) \to (f(u_1), f(u_4)) = (v_1, v_4) :$ *Valid edge in* G_2
- $(u_2, u_3) \to (f(u_2), f(u_3)) = (v_2, v_3) :$ *Valid edge in* G_2
- $(u_6, u_5) \to (f(u_6), f(u_5)) = (v_5, v_6) :$ *Valid edge in* G_2

Since f is a bijection from the vertex set of G_1 to the vertex set of G_2 that preserves all adjacency relationships perfectly, the function f is a valid isomorphism. Therefore, the two graphs are mathematically identical. $\square$

Question-5

Determine whether the given pair of directed graphs are isomorphic.



Solution: To determine if the directed graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are isomorphic, we compare their structural properties and invariants.

An isomorphism must preserve the out-degree (d_{out}) and in-degree (d_{in}) for corresponding vertices. Let us list the degree pairs (d_{out}, d_{in}) for each vertex:

Graph G_1 (vertices u_i):

- $u_1 : (1, 2)$
- $u_2 : (3, 2)$ (includes self-loop)
- $u_3 : (3, 1)$
- $u_4 : (0, 2)$

Graph G_2 (vertices v_i):

- $v_1 : (1, 1)$
- $v_2 : (3, 1)$
- $v_3 : (1, 2)$
- $v_4 : (2, 3)$ (includes self-loop)

We observe the following discrepancies between the two graphs:

1. **Existence of a Sink:** In graph G_1, vertex u_4 is a sink node with an out-degree of 0. In graph G_2, there is no vertex with an out-degree of 0.
2. **In-degree Maximum:** The maximum in-degree in G_1 is $\Delta_{in}(G_1) = 2$. However, in G_2, vertex v_4 has an in-degree of 3.
3. **Self-loop Properties:** In G_1, the vertex with a self-loop (u_2) has degrees $(3, 2)$. In G_2, the vertex with a self-loop (v_4) has degrees $(2, 3)$. For the graphs to be isomorphic, these degree pairs must match.

Because the graphs do not share the same degree sequences and structural properties (such as sink nodes and maximum in-degrees), there is no bijection $f : V_1 \to V_2$ that preserves adjacency.

Therefore, the two directed graphs are not isomorphic. Solution: To determine whether the two directed graphs are isomorphic, we can evaluate their graph invariants. Specifically, we will look at the out-degrees (the number of directed edges originating from a vertex) for each node. For two directed graphs to be isomorphic, their vertices must share the exact same out-degree sequence.

Let us calculate the out-degree, denoted as $\deg^+(x)$, for each vertex in the first graph:

- **Vertex u_1:** 1 outgoing edge (to u_4) $\implies \deg^+(u_1) = 1$
- **Vertex u_2:** 3 outgoing edges (to u_1, u_3, u_4) $\implies \deg^+(u_2) = 3$
- **Vertex u_3:** 1 outgoing edge (to u_1) $\implies \deg^+(u_3) = 1$
- **Vertex u_4:** 1 outgoing edge (to u_3) $\implies \deg^+(u_4) = 1$

The out-degree sequence for G_1 is $(3, 1, 1, 1)$.

Now, we calculate the out-degrees for the vertices in the second graph. Pay close attention to the direction of the bottom edge, which points from v_3 to v_2:

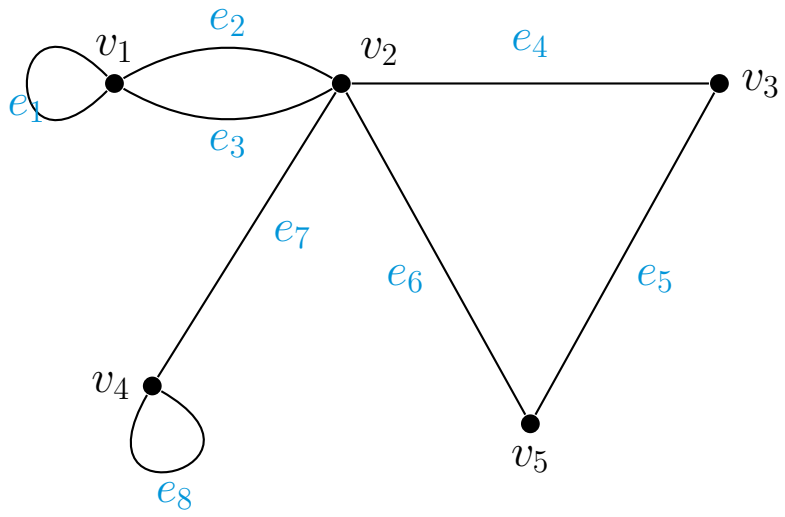
- **Vertex v_1:** 1 outgoing edge (to v_4) $\implies \deg^+(v_1) = 1$
- **Vertex v_2:** 2 outgoing edges (to v_1, v_4) $\implies \deg^+(v_2) = 2$
- **Vertex v_3:** 2 outgoing edges (to v_1, v_2) $\implies \deg^+(v_3) = 2$
- **Vertex v_4:** 1 outgoing edge (to v_3) $\implies \deg^+(v_4) = 1$

The out-degree sequence for G_2 is $(2, 2, 1, 1)$.

The first graph contains a vertex (u_2) with an out-degree of 3. The second graph lacks any vertex with an out-degree of 3. Because a graph isomorphism must preserve degree sequences, it is impossible to map the vertices of G_1 to G_2. Therefore, the two directed graphs are **not isomorphic**.

Question-7

Represent the following pseudograph using an incidence matrix.



Solution: An incidence matrix of an undirected graph is a matrix M where the rows represent the vertices and the columns represent the edges. The entries m_{ij} are defined as:

- $m_{ij} = 1$ if edge e_j is incident to vertex v_i.
- $m_{ij} = 0$ otherwise.

Note: For loops, the edge is incident to only one vertex, resulting in a single 1 in that edge's column.

Based on the provided graph, we have:

- **Vertices** $(n = 5)$**:** v_1, v_2, v_3, v_4, v_5
- **Edges** $(m = 8)$**:** $e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8$

This means our incidence matrix M will be a 5×8 matrix.

We determine the non-zero entries for each column (edge):

- e_1: Loop at $v_1 \implies$ Incident to v_1
- e_2: Connects v_1 and $v_2 \implies$ Incident to v_1, v_2
- e_3: Connects v_1 and $v_2 \implies$ Incident to v_1, v_2
- e_4: Connects v_2 and $v_3 \implies$ Incident to v_2, v_3
- e_5: Connects v_3 and $v_5 \implies$ Incident to v_3, v_5
- e_6: Connects v_2 and $v_5 \implies$ Incident to v_2, v_5
- e_7: Connects v_2 and $v_4 \implies$ Incident to v_2, v_4

- e_8: Loop at $v_4 \implies$ Incident to v_4

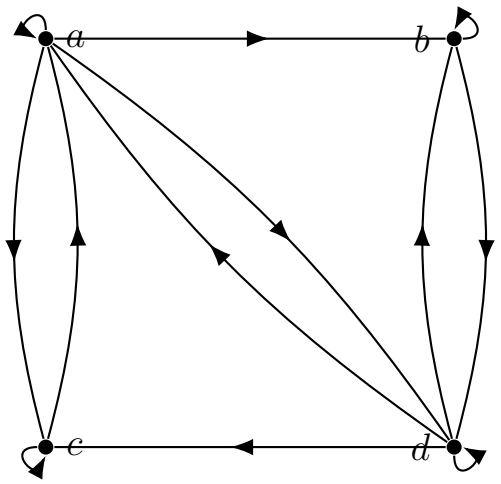
Putting it all together, we label the rows $v_1 \dots v_5$ and the columns $e_1 \dots e_8$ to yield the final incidence matrix:

$$M = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{array} \begin{array}{c} \begin{array}{cccccccc} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 & e_7 & e_8 \end{array} \\ \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{pmatrix} \end{array}$$

□

Question-8

Find the adjacency matrix of the given directed multigraph with respect to the vertices listed in alphabetic order.



Solution: An adjacency matrix A for a directed graph with n vertices is an $n \times n$ matrix where the entry $A_{ij} = 1$ if there is an edge pointing from vertex i to vertex j, and $A_{ij} = 0$ otherwise.

Let us construct the matrix by observing the outgoing edges from each vertex. We will order the rows and columns as a, b, c, d:

- **Vertex *a*:** There are outgoing edges to a (loop), b (straight edge), c (curved edge), and d (curved edge).
  Therefore, the first row is $1, 1, 1, 1$.

- **Vertex *b*:** There are outgoing edges to b (loop) and d (curved edge). There are no arrows pointing from b to a or c.
  Therefore, the second row is $0, 1, 0, 1$.

- **Vertex *c*:** There are outgoing edges to a (curved edge) and c (loop). There are no arrows pointing from c to b or d.
  Therefore, the third row is $1, 0, 1, 0$.

- **Vertex *d*:** There are outgoing edges to a (curved edge), b (curved edge), c (straight edge), and d (loop).
  Therefore, the fourth row is $1, 1, 1, 1$.

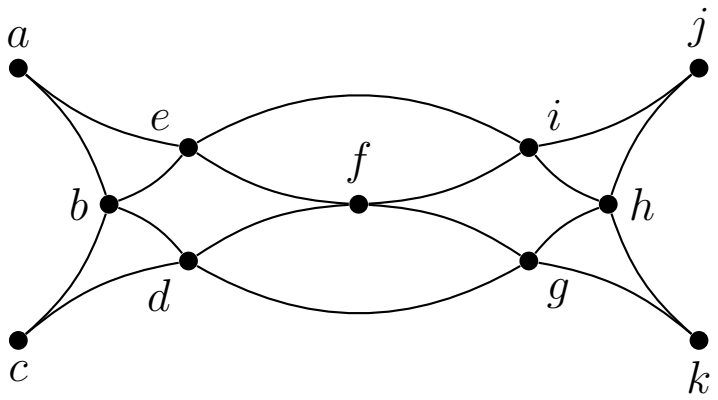
Combining these rows, the resulting adjacency matrix A is:

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

□

Question-9

Find Hamilton circuit or path for the following graph, if exists.



Solution: To determine if a Hamiltonian circuit exists, we examine the degrees of the vertices. A fundamental rule for finding Hamiltonian circuits is that if a vertex has a degree of exactly 2, both of its incident edges must be included in any Hamiltonian circuit.

Observe the outer vertices a, c, j, and k. They all have a degree of 2.

- $\deg(a) = 2 \implies$ edges $\{a, e\}$ and $\{a, b\}$ **must** be in the circuit.

- $\deg(c) = 2 \implies$ edges $\{c, b\}$ and $\{c, d\}$ **must** be in the circuit.

- $\deg(j) = 2 \implies$ edges $\{j, i\}$ and $\{j, h\}$ **must** be in the circuit.
- $\deg(k) = 2 \implies$ edges $\{k, g\}$ and $\{k, h\}$ **must** be in the circuit.

A vertex in a Hamiltonian circuit can only have exactly 2 incident edges used.

- Look at vertex b. In our circuit, we must use edges $\{a, b\}$ and $\{c, b\}$. Since b's quota of 2 edges is full, the other edges connected to b (which are $\{b, e\}$ and $\{b, d\}$) **cannot** be part of the circuit.
- Similarly, look at vertex h. We must use edges $\{j, h\}$ and $\{k, h\}$. Therefore, the other edges connected to h ($\{h, i\}$ and $\{h, g\}$) **cannot** be part of the circuit.

By forcing these edges, we have formed two required strings of vertices:

- **Path 1:** $e \to a \to b \to c \to d$
- **Path 2:** $i \to j \to h \to k \to g$

To form a full Hamiltonian circuit, we must connect these two paths along with the only remaining unvisited vertex, f. The available open endpoints of our paths are e, d and i, g.

- If we connect $e \to f \to i$, we successfully include vertex f and merge the top parts of our paths.
- Now we must close the circuit by connecting the bottom endpoints d and g. The edge $\{d, g\}$ exists in the graph (it is the long bottom arc).

Tracing this complete route yields:

$$a \to b \to c \to d \to g \to k \to h \to j \to i \to f \to e \to a$$

Checking this sequence, it visits all 11 vertices $(a, b, c, d, e, f, g, h, i, j, k)$ exactly once and returns to the starting vertex. All traversed edges exist in the original graph.

Yes, the graph contains a **Hamiltonian Circuit** (and consequently, it also contains multiple Hamiltonian Paths). $\square$

Question-10

Draw an undirected graph represented by the given adjacency matrix.

$$\begin{bmatrix} 0 & 1 & 3 & 0 & 4 \\ 1 & 2 & 1 & 3 & 0 \\ 3 & 1 & 1 & 0 & 1 \\ 0 & 3 & 0 & 0 & 2 \\ 4 & 0 & 1 & 2 & 3 \end{bmatrix}$$

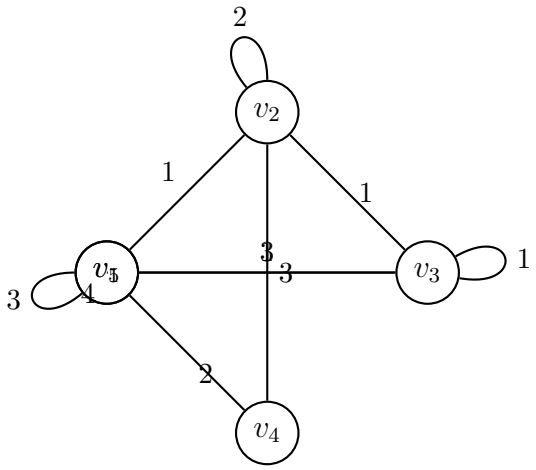
Solution: We are asked to draw the undirected graph it represents. We'll identify this as an adjacency matrix for a weighted, undirected multigraph with self-loops. Let's list the vertices as $V = \{v_1, v_2, v_3, v_4, v_5\}$.

The matrix entries correspond to the edge weights between vertices:

- Vertices: Since it is a 5×5 matrix, there are 5 vertices.
- Undirected: The matrix is symmetric ($M = M^T$), so the graph is undirected.
- Weighted Edges: Entry M_{ij} (where $i \neq j$) represents the weight of the single edge between v_i and v_j. A non-zero entry M_{ii} indicates a single self-loop with that weight on vertex v_i.

Here is the full list of vertices and edges with their weights, derived from the matrix:

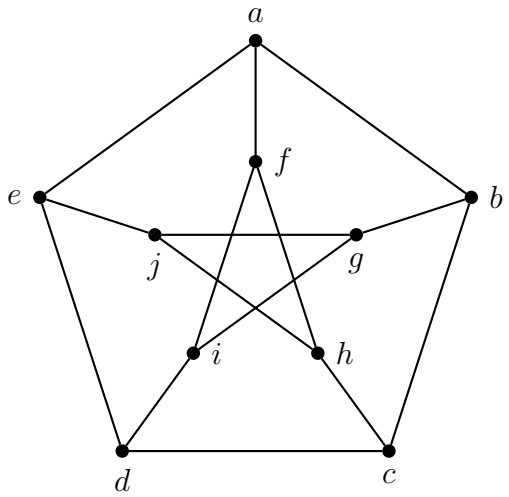
- Vertex Set: $V = \{v_1, v_2, v_3, v_4, v_5\}$
- Edge Set with Weights:
  - (v_1, v_2) with weight 1
  - (v_1, v_3) with weight 3
  - (v_1, v_5) with weight 4
  - (v_2, v_3) with weight 1
  - (v_2, v_4) with weight 3
  - (v_3, v_5) with weight 1
  - (v_4, v_5) with weight 2
  - Self-loop on v_2 with weight 2
  - Self-loop on v_3 with weight 1
  - Self-loop on v_5 with weight 3



□

Question-11

Determine whether the following graph has an Euler circuit? If not, find an Euler path. For the same graph, find for Hamilton path or circuit.



Solution: The graph displays the classic **Petersen graph**. Let V be the set of vertices and E be the set of edges. The graph has $|V| = 10$ vertices and $|E| = 15$ edges. By observing the graph, we can see that every vertex is connected to exactly 3 edges. Thus, it is a 3-regular (cubic) graph.

A connected graph contains an Euler circuit if and only if every vertex in the graph has an even degree. In our graph, the degree of every vertex is 3 (an odd number). Because there are vertices with an odd degree, the graph **does not have an Euler circuit**.

A connected graph contains an Euler path if and only if it has exactly zero or exactly two vertices of odd degree. As established, all 10 vertices in this graph have a degree of 3. Since there are more than two vertices with an odd degree, the graph **does not have an Euler path**.

A Hamilton circuit is a cycle that visits every vertex exactly once and returns to the starting vertex. The Petersen graph is a famous example of a hypohamiltonian graph, meaning it **does not contain a Hamilton circuit**. If you remove any single vertex from it, the remaining graph becomes Hamiltonian, but the full 10-vertex graph cannot form a complete cycle.

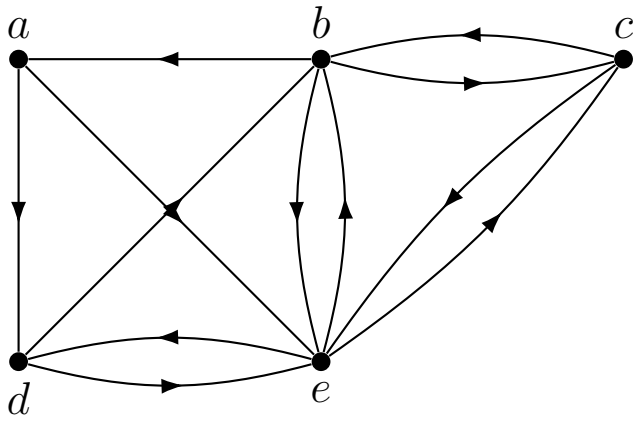
A Hamilton path is a path that visits every vertex in the graph exactly once without needing to return to the origin. The Petersen graph **does have a Hamilton path**. To find one, we can traverse the outer pentagon, cross over to the inner star, and trace through the remaining vertices. A valid sequence of vertices for a Hamilton path is:

$$a \to b \to c \to d \to e \to j \to g \to i \to f \to h$$

□

Question-12

Determine whether the following directed graph has an Euler circuit. Construct an Euler circuit if one exists. If no Euler circuit exists, determine whether the directed graph has an Euler path. Construct the Euler path if exists.



Solution: To determine whether the given directed graph has an Euler circuit or an Euler path, we must evaluate the in-degrees ($\deg^-$) and out-degrees ($\deg^+$) of every vertex. Let $V = \{a, b, c, d, e\}$.

By counting the incoming and outgoing arrows for each node, we get:

- **Vertex *a***:
  - In-edges: from $b \Rightarrow \deg^-(a) = 1$
  - Out-edges: to $d, e \Rightarrow \deg^+(a) = 2$
- **Vertex *b***:
  - In-edges: from $c, d, e \Rightarrow \deg^-(b) = 3$
  - Out-edges: to $a, c, e \Rightarrow \deg^+(b) = 3$
- **Vertex *c***:
  - In-edges: from $b, e \Rightarrow \deg^-(c) = 2$
  - Out-edges: to $b, e \Rightarrow \deg^+(c) = 2$
- **Vertex *d***:
  - In-edges: from $a, e \Rightarrow \deg^-(d) = 2$
  - Out-edges: to $b, e \Rightarrow \deg^+(d) = 2$
- **Vertex *e***:

– In-edges: from $a, b, c, d \Rightarrow \deg^-(e) = 4$

– Out-edges: to $b, c, d \Rightarrow \deg^+(e) = 3$

A directed graph contains an Euler circuit if and only if it is weakly connected and $\deg^-(v) = \deg^+(v)$ for every vertex $v \in V$. Since $\deg^-(a) \neq \deg^+(a)$ and $\deg^-(e) \neq \deg^+(e)$, **no Euler circuit exists**.

A directed graph contains an Euler path (but no circuit) if and only if it is weakly connected and:

1. Exactly one vertex has $\deg^+(v) - \deg^-(v) = 1$ (Start vertex).
2. Exactly one vertex has $\deg^-(v) - \deg^+(v) = 1$ (End vertex).
3. All other vertices have equal in-degrees and out-degrees.

Based on our analysis:

- For vertex a: $\deg^+(a) - \deg^-(a) = 2 - 1 = 1$.
- For vertex e: $\deg^-(e) - \deg^+(e) = 4 - 3 = 1$.
- For vertices b, c, d: $\deg^-(v) = \deg^+(v)$.

Since these conditions are met perfectly, **an Euler path exists**. It must start at vertex a and end at vertex e.

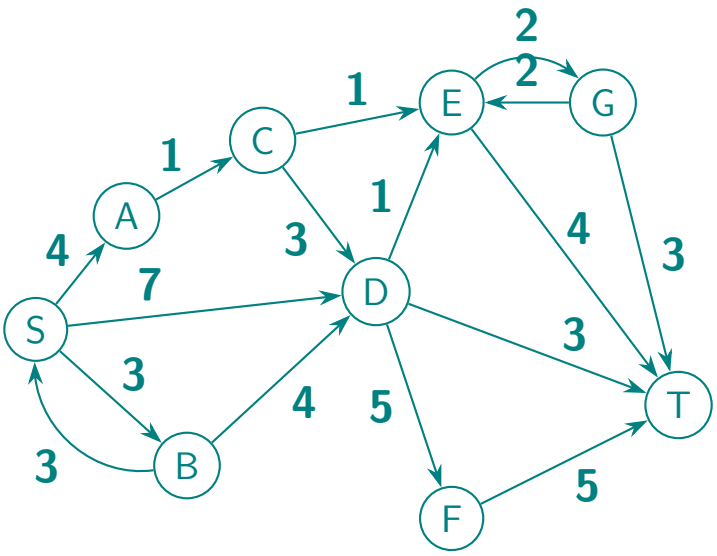
Starting at vertex a and ending at vertex e, we can traverse all 12 distinct edges of the graph exactly once. One such valid Euler path is:

$$a \to d \to b \to a \to e \to d \to e \to b \to c \to b \to e \to c \to e$$

This traverses the edges in the following order: $(a, d), (d, b), (b, a), (a, e), (e, d), (d, e), (e, b), (b, c), (c, b), (b, e), (e, c), (c, e)$. □

Question-13

Consider the following directed graph. Find the shortest path between the vertices S and T of the graph by Dijkstra's algorithm.



Solution: Let $d(v)$ be the shortest distance from the source node S to node v. Initially, $d(S) = 0$ and $d(v) = \infty$ for all other vertices.

The table below tracks the shortest distance to each node at every step. In each row, the unvisited node with the smallest tentative distance is selected and its neighbors are updated.

Selected Node	**S**	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**T**
Initial	0	∞	∞	∞	∞	∞	∞	∞	∞
S $(d = 0)$	–	**4**	**3**	∞	**7**	∞	∞	∞	∞
B $(d = 3)$	–	4	–	∞	7	∞	∞	∞	∞
A $(d = 4)$	–	–	–	**5**	7	∞	∞	∞	∞
C $(d = 5)$	–	–	–	–	7	**6**	∞	∞	∞
E $(d = 6)$	–	–	–	–	7	–	∞	**8**	**10**
D $(d = 7)$	–	–	–	–	–	–	**12**	8	10
G $(d = 8)$	–	–	–	–	–	–	12	–	10
T $(d = 10)$	–	–	–	–	–	–	12	–	–
F $(d = 12)$	–	–	–	–	–	–	–	–	–

The shortest distance from S to T is **10**.

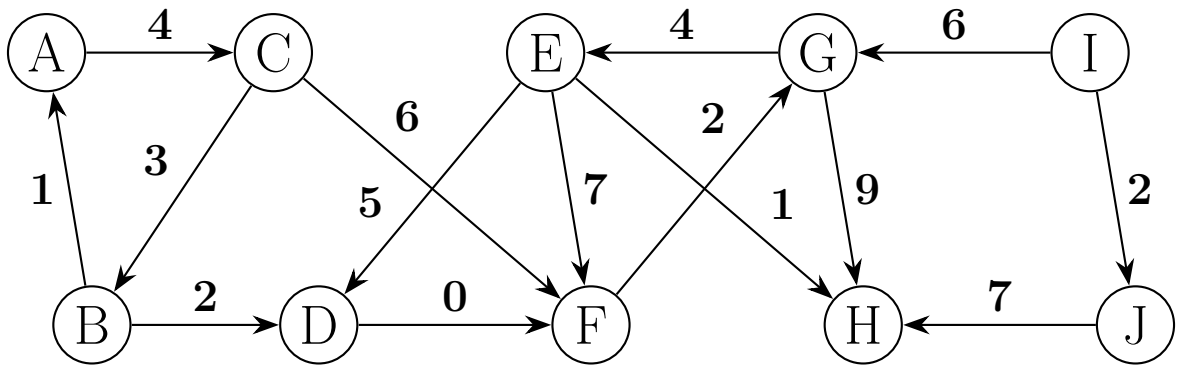
Tracing back the distance updates reveals that there are three valid shortest paths that all result in a total weight of 10:

- **Path 1:** $S \to A \to C \to E \to T$ (Weights: $4 + 1 + 1 + 4 = 10$)
- **Path 2:** $S \to D \to T$ (Weights: $7 + 3 = 10$)
- **Path 3:** $S \to B \to D \to T$ (Weights: $3 + 4 + 3 = 10$)

□

Question-14

Let G be the directed graph given below. Use Dijkstra's algorithm to find the shortest path between the vertices A and H.



Solution: To find the shortest path, we initialize the distance to the source node A as $d(A) = 0$ and all other nodes as ∞. We then iteratively relax the edges.

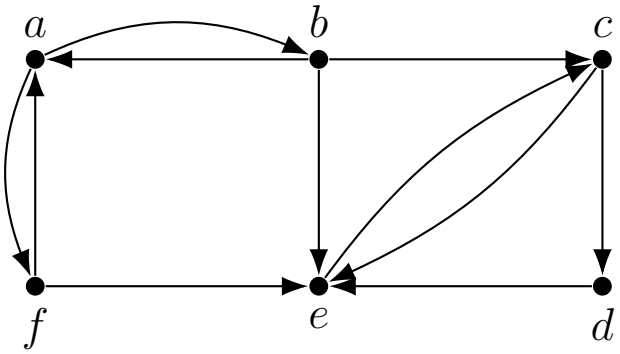
Step	Node	Neighbors Relaxed	Distance Vector (A, B, C, D, E, F, G, H)
0	—	—	$(0, \infty, \infty, \infty, \infty, \infty, \infty, \infty)$
1	A	$C : 0 + 4 = 4$	$(0, \infty, 4, \infty, \infty, \infty, \infty, \infty)$
2	C	$B : 4 + 3 = 7, F : 4 + 6 = 10$	$(0, 7, 4, \infty, \infty, 10, \infty, \infty)$
3	B	$D : 7 + 2 = 9$	$(0, 7, 4, 9, \infty, 10, \infty, \infty)$
4	D	$F : 9 + 0 = 9$	$(0, 7, 4, 9, \infty, 9, \infty, \infty)$
5	F	$G : 9 + 2 = 11$	$(0, 7, 4, 9, \infty, 9, 11, \infty)$
6	G	$E : 11 + 4 = 15, H : 11 + 9 = 20$	$(0, 7, 4, 9, 15, 9, 11, 20)$
7	E	$H : 15 + 1 = 16$	$(0, 7, 4, 9, 15, 9, 11, 16)$
8	H	Goal Reached	$(0, 7, 4, 9, 15, 9, 11, 16)$

The shortest path is determined by following the predecessors of the relaxed distances:

- **Path:** $A \to C \to B \to D \to F \to G \to E \to H$
- **Total Weight:** $4 + 3 + 2 + 0 + 2 + 4 + 1 = 16$

Question-15

Find the strongly connected components of the given digraph.



Solution: A **strongly connected component (SCC)** of a directed graph is a maximal subgraph in which for every pair of vertices u and v, there is a directed path from u to v and a directed path from v to u.

To find the SCCs of the given digraph, we group the vertices into sets where all vertices in a set are mutually reachable:

- Observe the vertices on the left. There is a two-way connection between a and b ($a \to b$ and $b \to a$), meaning they can reach each other.
- Similarly, there is a two-way connection between a and f ($a \to f$ and $f \to a$).
- By transitivity, b can reach f (via a) and f can reach b (via a). Thus, a, b, and f are all mutually reachable.
- We must check if this component can be expanded. The vertices b and f have outgoing edges to the right side of the graph ($b \to c$, $b \to e$, $f \to e$). However, there are **no returning edges** from $\{c, d, e\}$ back to $\{a, b, f\}$.
- Therefore, no other vertices can be part of this mutual-reachability group. The set $\{a, b, f\}$ is a maximal strongly connected component.
- Now look at the remaining vertices on the right. There is a two-way connection between c and e ($c \to e$ and $e \to c$).
- We also have a path passing through d. There is an edge $c \to d$ and an edge $d \to e$.
- This creates a directed cycle: $c \to d \to e \to c$. Because they form a continuous loop, any vertex in $\{c, d, e\}$ can reach any other vertex in that same set.
- Since there are no other vertices left in the graph to check, this forms our second maximal strongly connected component.

The digraph has exactly two strongly connected components:

$$C_1 = \{a, b, f\}$$

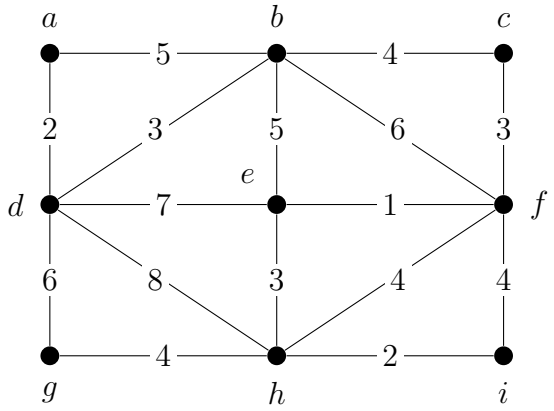
$$C_2 = \{c, d, e\}$$

Step	Visited Vertices	Available Connecting Edges (Weight)	Chosen Edge	V
1	$\{a\}$	$(a,b)=5$, $(a,d)=2$	(a,d)	
2	$\{a,d\}$	$(a,b)=5$, $(d,b)=3$, $(d,e)=7$, $(d,g)=6$, $(d,h)=8$	(d,b)	
3	$\{a,d,b\}$	$(b,c)=4$, $(b,e)=5$, $(b,f)=6$, $(d,e)=7$, ...	(b,c)	
4	$\{a,d,b,c\}$	$(c,f)=3$, $(b,e)=5$, $(b,f)=6$, $(d,e)=7$, ...	(c,f)	
5	$\{a,d,b,c,f\}$	$(f,e)=1$, $(f,i)=4$, $(f,h)=4$, $(b,e)=5$, ...	(f,e)	
6	$\{a,d,b,c,f,e\}$	$(e,h)=3$, $(f,i)=4$, $(f,h)=4$, $(d,g)=6$, ...	(e,h)	
7	$\{a,d,b,c,f,e,h\}$	$(h,i)=2$, $(h,g)=4$, $(f,i)=4$, $(d,g)=6$	(h,i)	
8	$\{a,d,b,c,f,e,h,i\}$	$(h,g)=4$, $(d,g)=6$	(h,g)	

Table 1: Execution of Prim's Algorithm starting from vertex a.

Question-16

Use Prim's algorithm to find a minimum spanning tree for the given weighted graph.



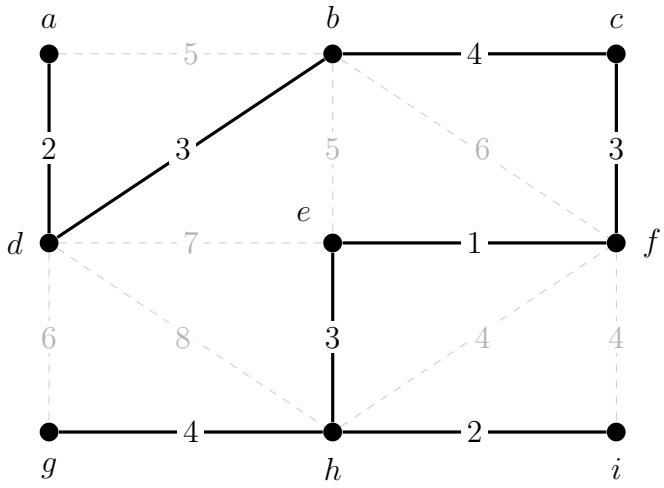
Solution: Prim's algorithm builds a Minimum Spanning Tree (MST) by starting at an arbitrary vertex and continually adding the lowest-weight edge that connects a visited vertex to an unvisited vertex. For this solution, we will arbitrarily start at vertex **a**.

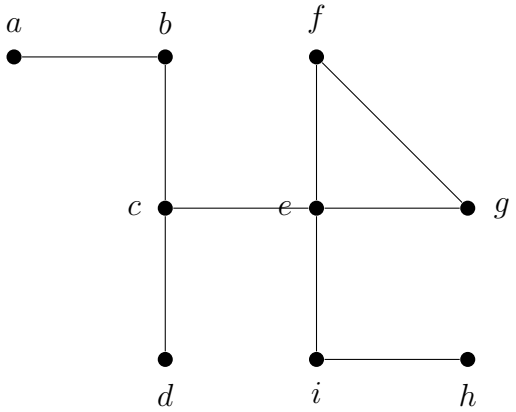
Total Minimum Weight: $2+3+4+3+1+3+2+4=$ **22**

The resulting Minimum Spanning Tree is illustrated below. The bold black lines represent the edges included in the MST, while the dashed gray lines represent the unused edges from the original graph.

Question-17

Find all the cut vertices and cut set of the given graph.





Solution: A vertex v is a cut vertex if $G - \{v\}$ has more connected components than G. By inspection:

- **Vertex** b: Separates a from the rest of the graph.
- **Vertex** c: Separates d and the block $\{a, b\}$ from the rest.
- **Vertex** e: Separates the triangle/right-side structure from the left-side structure and the bottom tail $\{i, h\}$.
- **Vertex** i: Separates h from the rest of the graph.

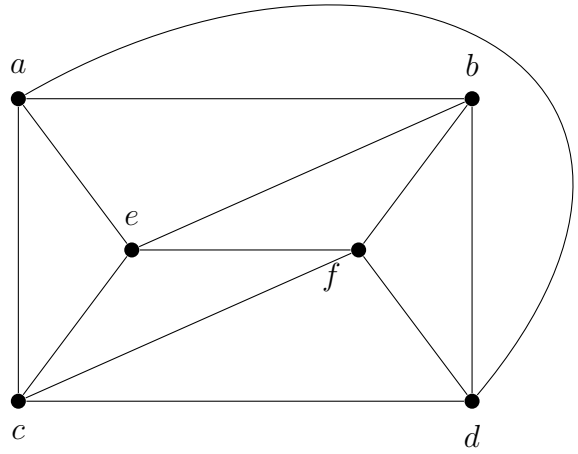
Cut Vertices: $V_{cut} = \{b, c, e, i\}$

A cut set is a minimal set of edges whose removal increases the number of connected components.

- **Bridges (Single-edge cut sets):** $\{\{a, b\}, \{b, c\}, \{c, d\}, \{c, e\}, \{e, i\}, \{i, h\}\}$
- **Minimal multi-edge cut sets:**
  - $\{\{e, f\}, \{f, g\}\}$ (separates vertex f)
  - $\{\{e, g\}, \{f, g\}\}$ (separates vertex g)
  - $\{\{e, f\}, \{e, g\}\}$ (separates the $f - g$ edge from e)

Question-18

Find the chromatic number and the chromatic index of the given graph.



Solution: The chromatic number is the minimum number of colors required to color the vertices such that no two adjacent vertices share the same color.

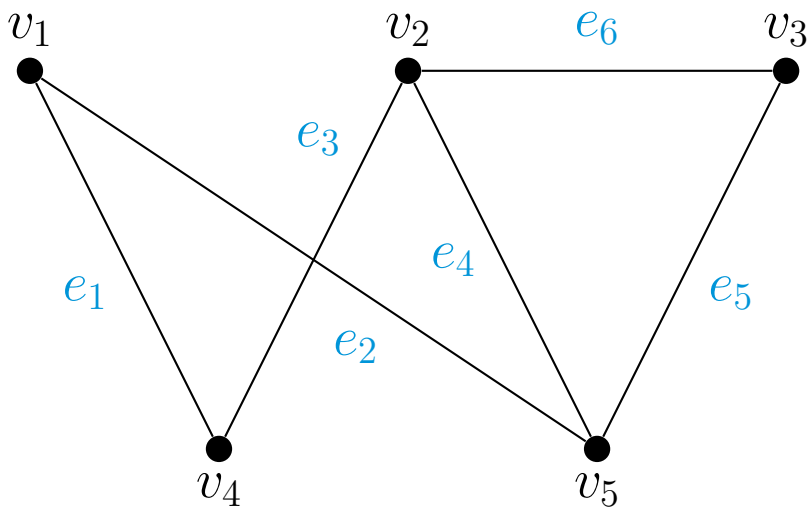
- **Lower Bound:** The graph contains several 3-cliques (triangles), such as $\{a, b, e\}$. Thus, $\chi(G) \geq 3$.
- **Upper Bound:** We can provide a 3-coloring:
  - Color 1: $\{a, f\}$
  - Color 2: $\{b, c\}$
  - Color 3: $\{d, e\}$
- **Conclusion:** Since a 3-coloring exists and $\chi(G) \geq 3$, we have $\chi(G) = 3$.

The chromatic index is the minimum number of colors required to color the edges such that no two edges incident to the same vertex share a color.

- **Maximum Degree:** By counting the edges incident to each vertex: $\deg(a) = 4, \deg(b) = 4, \deg(c) = 4, \deg(d) = 4, \deg(e) = 4, \deg(f) = 4$. Thus, $\Delta(G) = 4$.
- **Vizing's Theorem:** $\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1$. So $\chi'(G)$ is either 4 or 5.
- **Conclusion:** For this 4-regular graph with 6 vertices, the edges can be partitioned into 4 perfect matchings. Therefore, $\chi'(G) = \Delta(G) = 4$.

12 Excercise

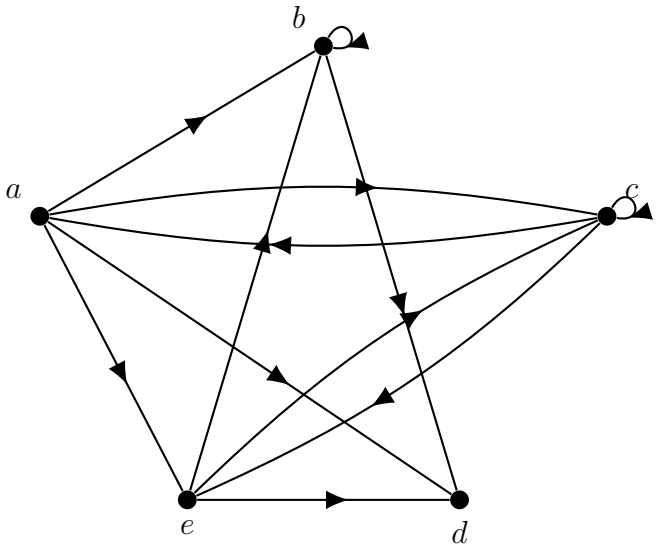
1. Represent the following graph by an incidence matrix.



Answer:

$$\begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

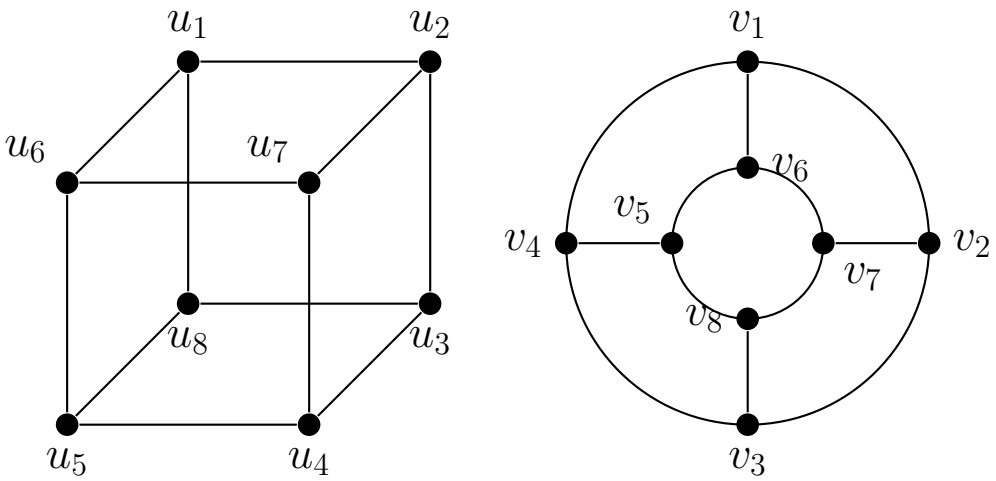
2. Represent the following directed graph by an adjacency matrix.



Answer:

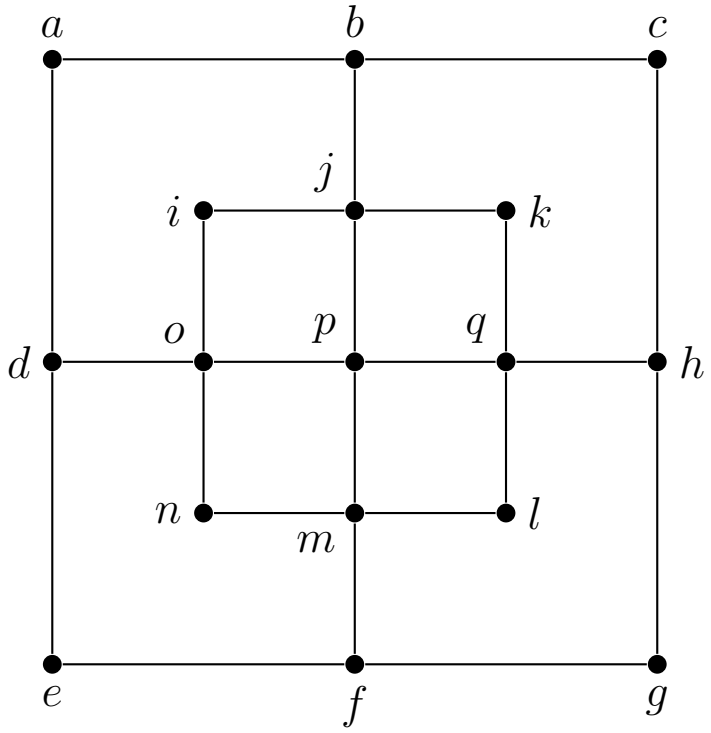
$$\begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 \end{bmatrix}$$

3. Use paths either to show the following pair of graphs are not isomorphic or find an isomorphism between them.



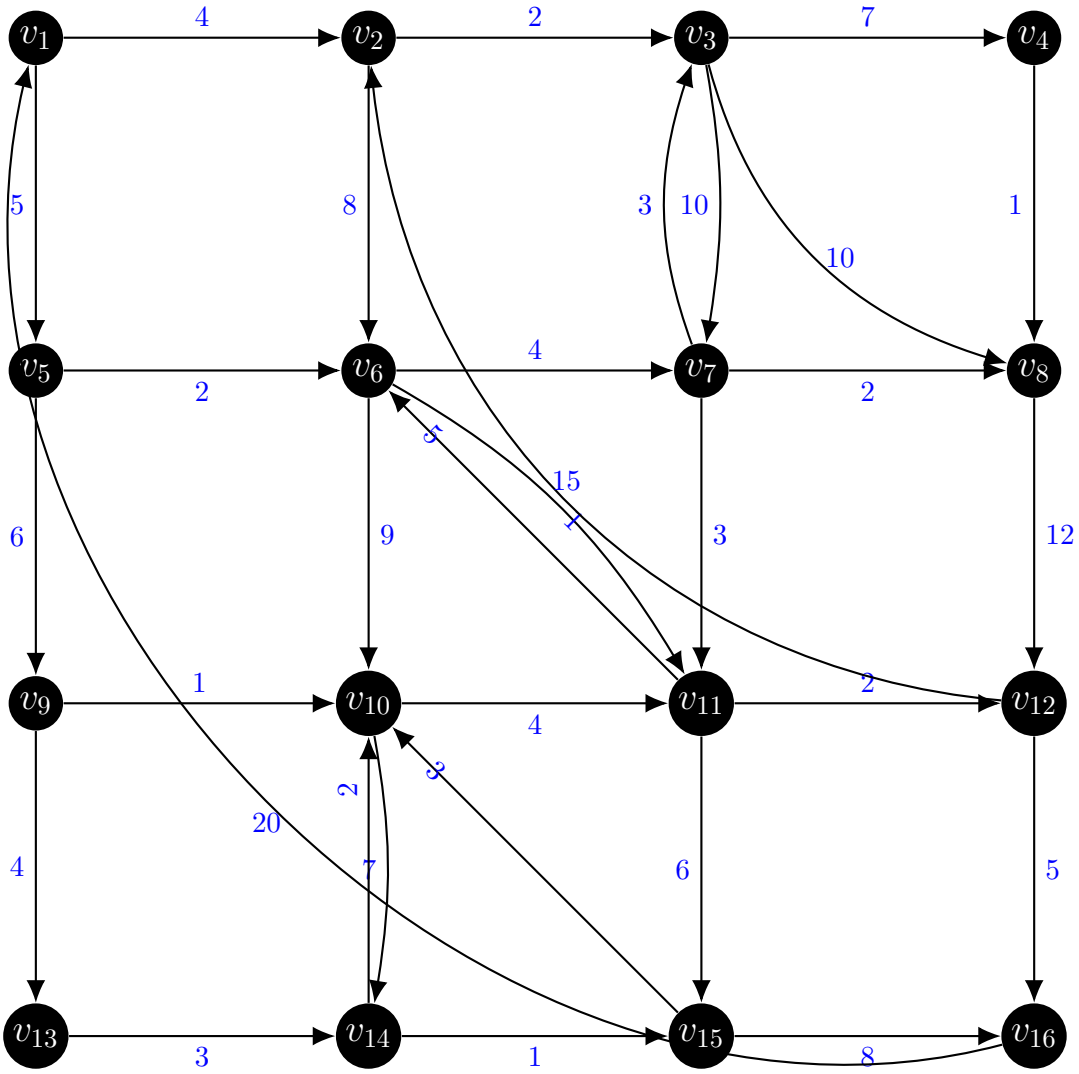
Answer: Isomorphic.

4. Determine whether the following graph has an Euler circuit? If not, find an Euler path. For the same graph, find for Hamilton path or circuit.



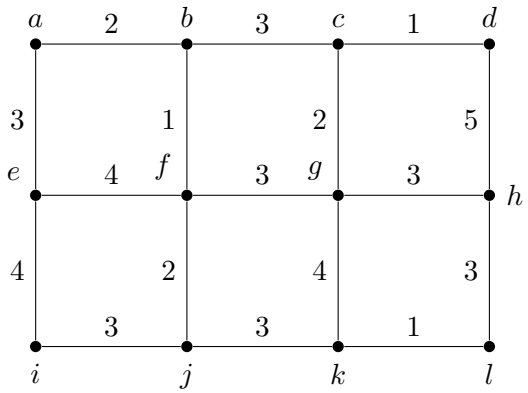
Answer: The graph has neither an Euler circuit nor an Euler path. This graph has neither a Hamilton path nor a Hamilton circuit.

5. Using Dijkstra's Algorithm, determine the shortest distance and the shortest path between the vertices v_1 to v_{16}.



Answer: The shortest distane is 15 amd the shortest path is $v_1 \xrightarrow{5} v_5 \xrightarrow{2} v_6 \xrightarrow{1} v_{11} \xrightarrow{2} v_{12} \xrightarrow{5} v_{16}$.

6. Use Prim's algorithm to find a minimum spanning tree for the given weighted graph.



Answer: Total weight of the MST is 24.